

From Noise to Signal: Leveraging Discriminative Model Identifiers for Efficient Product Blocking

David von Schledorn Márquez

Erasmus University Rotterdam, Burgemeester Oudlaan 50, 3062 PA Rotterdam, the Netherlands

4davidvam@gmail.com

Abstract. Duplicate detection is critical yet expensive in e-commerce. Standard Locality-Sensitive Hashing (LSH) often suffers low precision on noisy product titles. We propose an enhanced pipeline integrating *Model Word Weighting* to prioritize alphanumeric identifiers and *Stop Shingle Pruning* to filter high-frequency noise. Experiments on real-world television listings demonstrate that our approach increases the F_1 -measure by over 14% compared to a standard baseline. Furthermore, it achieves this with high efficiency, requiring only 1.2% of theoretical pairwise comparisons compared to 58.1% for the baseline. These results underscore the importance of domain-specific feature engineering in scalable entity resolution.

Keywords: Duplicate Detection, Locality-Sensitive Hashing, Entity Resolution, E-commerce, Feature Engineering

1 Introduction

Duplicate detection is critical for e-commerce but computationally expensive ($O(N^2)$). While blocking techniques like Locality-Sensitive Hashing (LSH) improve scalability, standard implementations often suffer low precision on structured data (e.g., televisions). Common terms (e.g., "Inch") create noise, while discriminative model codes are often undervalued.

We propose an enhanced pipeline to address this, evaluating two specific additions:

1. **Model Word Weighting:** We up-weight alphanumeric tokens acting as unique identifiers.
2. **Stop Shingle Pruning:** We filter high-frequency substrings to sharpen LSH blocking.

Experiments demonstrate that our approach increases the F_1 -measure by around 15% compared to standard LSH. Furthermore, it achieves high efficiency, performing only 1.2% of theoretical pairwise comparisons compared to the baseline's 58.1%.

The implementation of our pipeline is available online¹.

¹ <https://github.com/DvSM-git/Discriminative-Identifiers-for-Product-Blocking>

2 Related Work

Duplicate detection (Entity Resolution) is vital for aggregating e-commerce data from diverse sources, with research primarily focusing on mitigating the quadratic complexity inherent in identifying duplicates among heterogeneous records.

To avoid exhaustive pair-wise comparison, Broder et al. introduced *Min-hashing* [1], a technique that rapidly estimates Jaccard similarity by reducing documents to compact signatures. Building on this, Indyk and Motwani proposed *Locality-Sensitive Hashing* (LSH) [3], which addresses high dimensionality by hashing similar items into the same "buckets" with high probability. This blocking technique drastically reduces the search space, enabling scalable nearest-neighbor search.

However, generic similarity measures often fail on structured television specifications. Van Bezu et al. addressed this with the *Multi-Component Similarity Method* (MSM) [4], which computes similarity based on decomposed product attributes rather than raw text. While precise, MSM is computationally expensive. To improve scalability, Van Dam et al. proposed MSMP [5], integrating LSH as a pre-selection step to limit expensive MSM comparisons to candidate clusters. Most recently, Hartveld et al. extended this with MSMP+ [2]; by utilising highly discriminative "model words" to drive the hashing process, MSMP+ reduces false positives caused by noisy titles, thereby refining both the efficiency and precision of candidate generation.

3 Methodology

The pipeline employs MinHash and LSH for efficient candidate generation, followed by MSM for final classification. To isolate the impact of our "model word" and "stop shingle" optimisations, we benchmark our Full Model against a "Restricted Model" baseline devoid of these domain-specific features.

3.1 Data

We utilised a dataset of television listings extracted from four online retailers (www.amazon.com, www.bestbuy.com, www.newegg.com, and www.thenerds.com). The corpus is serialised in JSON format and contains five primary fields: product title, model identifier, URL, vendor, and a dynamic dictionary of specifications (Key-Value Pairs). Due to inconsistent retailer standards, the schema of these KVPs is non-uniform, resulting in sparse or varying attribute descriptions for identical entities across different sources.

3.2 Data Preprocessing

To capture both lexical overlap and precise attributes, we transform raw data into a hybrid feature set. First, text is normalised by lowercasing, standardising

units (e.g., *inches* $\rightarrow$ *inch*), and removing non-alphanumeric characters (except hyphens).

Feature extraction then proceeds via two parallel strategies. First, we implement shingling with pruning. The text is decomposed into character k -shingles. To reduce noise, common shingles are identified; those appearing in more than a threshold t_{freq} proportion of descriptions are pruned as "stop shingles." Next, we implement model word weighting. Alphanumeric identifiers (e.g., *UN40H6350*) are extracted via regex. To weight these within a set-based similarity framework, we use token replication, where each identifier is duplicated w times with unique suffixes (e.g., UN40H6350_01, UN40H6350_02), ensuring they heavily influence the subsequent hashing process. The baseline model does not contain any of these extra preprocessing steps.

3.3 Minhashing

MinHash compresses feature sets into fixed-length signatures while preserving Jaccard similarity:

$$\Pr[h_{\min}(A) = h_{\min}(B)] = J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (1)$$

Signature Generation Tokens are mapped to 32-bit integers X_S via CRC32. We then generate a signature of length K using random linear permutations over a Mersenne prime $p = 2^{32} - 1$. The k -th component is:

$$\Sigma_S[k] = \min_{x \in X_S} ((a_k \cdot x + b_k) \pmod{p}) \quad (2)$$

3.4 Locality-Sensitive Hashing (LSH)

To overcome the $O(N^2)$ complexity of exhaustive comparison, we employ the LSH Banding technique, pruning the search space to a linear subset.

Banding Strategy The MinHash signature matrix $M \in \mathbb{Z}^{K \times N}$ is partitioned into b bands of r rows ($K = b \times r$). Two documents form a "candidate pair" if their sub-vectors $\mathbf{v}_d^{(j)}$ are identical in at least one band:

$$\exists j : \mathbf{v}_{d_1}^{(j)} = \mathbf{v}_{d_2}^{(j)} \quad (3)$$

To detect these collisions efficiently, we hash each r -dimensional sub-vector to a scalar bucket index $H(\mathbf{v}_d^{(j)})$. This allows for $O(1)$ lookup, placing documents with identical sub-vectors into the same buckets.

Candidate Generation and Probability Candidates are generated by retrieving all pairs residing in the same buckets. The set of candidate pairs C is the union of collisions across all b bands. The probability $P(\text{candidate} \mid s)$ that a pair with Jaccard similarity s is retrieved is governed by the S-curve function:

$$P(\text{candidate} \mid s) = 1 - (1 - s^r)^b \quad (4)$$

This function creates a sharp threshold at $t \approx (1/b)^{1/r}$, efficiently filtering dissimilar pairs while minimising false negatives.

3.5 Multi-Component Similarity Method (MSM)

For candidate pairs, we compute a composite similarity Φ by aggregating structured and unstructured features. First, a blocking pass sets similarity to zero for same-shop pairs or mismatches in brand/resolution.

For valid pairs, we calculate three scores: (1) Sim_{attr} , derived from a greedy q-gram matching of Key-Value Pairs; (2) Sim_{mw} , the Jaccard index of alphanumeric model words extracted from unmatched keys; and (3) Sim_{title} , a hybrid of Cosine similarity and Levenshtein distance on title model identifiers.

The final similarity is a weighted linear combination:

$$\Phi(p_i, p_j) = \theta_1 Sim_{attr} + \theta_2 Sim_{mw} + \mu Sim_{title} \quad (5)$$

To resolve final entities, we apply Hierarchical Agglomerative Clustering (HAC) on the distance matrix $1 - \Phi$, merging clusters until an inter-cluster distance threshold ϵ is reached.

4 Evaluation

To isolate the impact of our hybrid feature engineering, we evaluate two distinct configurations:

- **Restricted Model (Baseline):** A standard LSH implementation using raw character shingling, lacking domain-specific optimisations.
- **Full Model (Proposed):** Integrates *Model Word Weighting* and *Stop Shingle Pruning* to maximise the signal-to-noise ratio.

Both models were tuned independently using Bayesian Optimisation (Optuna). We maximised the F_1 -measure by optimising shingle size, LSH parameters (b, r), and MSM weights over 5 bootstrap iterations (out-of-bag) to prevent overfitting.

4.1 Evaluation Metrics

We evaluate LSH candidate generation ($\mathcal{C}$) and final MSM clustering ($\mathcal{M}$) against the set of true duplicates ($\mathcal{D}$).

For LSH, we measure Pair Quality (PQ), Pair Completeness (PC), and their harmonic mean F_1^* . Efficiency is the ratio of candidates $|\mathcal{C}|$ to total pairs $N(N-1)/2$.

$$PQ = \frac{|\mathcal{C} \cap \mathcal{D}|}{|\mathcal{C}|}, \quad PC = \frac{|\mathcal{C} \cap \mathcal{D}|}{|\mathcal{D}|}, \quad F_1^* = \frac{2 \cdot PQ \cdot PC}{PQ + PC} \quad (6)$$

For final classification, we report standard Precision, Recall, and F_1 -measure based on true positives $TP = |\mathcal{M} \cap \mathcal{D}|$:

$$Precision = \frac{TP}{|\mathcal{M}|}, \quad Recall = \frac{TP}{|\mathcal{D}|}, \quad F_1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (7)$$

4.2 Results

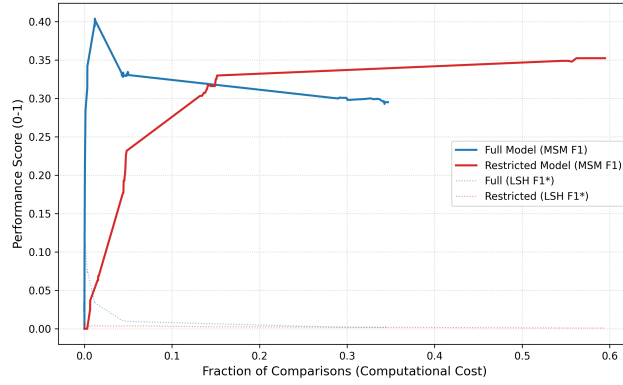


Fig. 1. Performance comparison: Full Model (blue) vs. Restricted Model (red). Solid lines denote final MSM F_1 ; dotted lines denote LSH candidate quality F_1^* .

Classification Effectiveness The Full Model outperforms the baseline, achieving a peak MSM F_1 score of **0.404**, compared to the Restricted Model’s maximum of **0.352**. This confirms that prioritising model identifiers enables high-precision resolution despite ambiguous titles.

Computational Efficiency The Full Model demonstrates superior scalability:

1. **Rapid Convergence:** It reaches peak effectiveness while performing only **1.2%** of theoretical comparisons.
2. **Baseline Inefficiency:** The Restricted Model requires processing **58.1%** of all pairs to reach its (lower) peak.

Candidate Quality The efficiency gap is driven by LSH quality (F_1^*). The Full Model peaks at $F_1^* = 0.117$ with minimal overhead (0.06% comparisons), whereas the Restricted Model's F_1^* remains negligible (peak 0.009). Overall, our optimisations improve the F_1 -measure by $\sim 15\%$ while reducing computational effort by a factor of nearly 25.

5 Conclusion

We addressed the challenge of scalable duplicate detection in the structured domain of consumer electronics. While Locality-Sensitive Hashing (LSH) reduces complexity, generic implementations often fail on product specifications. We proposed an enhanced pipeline integrating domain-specific feature engineering, specifically "model word" weighting and "stop shingle" pruning, into the MSMP framework.

Our evaluation on real-world TV listings demonstrates significant gains. The proposed Full Model achieved a peak F_1 -measure of 0.404, outperforming the baseline ($F_1 \approx 0.352$) by roughly 15%. Crucially, our optimisations enabled high-precision blocking, requiring only 1.2% of theoretical pairwise comparisons to reach optimal performance, whereas the baseline required over 58%.

These results underscore the value of hybrid approaches combining probabilistic blocking with semantic domain knowledge. While LSH ensures scalability, rule-based signals are essential for distinguishing similar product variants. Future work may explore applying this methodology to other e-commerce categories or integrating deep learning embeddings for improved title similarity.

References

1. Broder, A.Z.: On the resemblance and containment of documents. In: Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No. 97TB100171). pp. 21–29. IEEE (1997)
2. Hartveld, A., van Keulen, M., Mathol, D., van Noort, T., Plaatsman, T., Frasincar, F., Schouten, K.: An lsh-based model-words-driven product duplicate detection method. In: International Conference on Advanced Information Systems Engineering. pp. 409–423. Springer (2018)
3. Indyk, P., Motwani, R.: Approximate nearest neighbors: towards removing the curse of dimensionality. In: Proceedings of the thirtieth annual ACM symposium on Theory of computing. pp. 604–613 (1998)
4. Van Bezu, R., Borst, S., Rijkse, R., Verhagen, J., Vandic, D., Frasincar, F.: Multi-component similarity method for web product duplicate detection.(2015) (2015)
5. Van Dam, I., van Ginkel, G., Kuipers, W., Nijenhuis, N., Vandic, D., Frasincar, F.: Duplicate detection in web shops using lsh to reduce the number of computations. In: Proceedings of the 31st Annual ACM Symposium on Applied Computing. pp. 772–779 (2016)